MongoDB_Quick_Revision_Short_Note

https://gemini.google.com/app/0f5179a241dde299

*User prompt: Solve All The Question and give me All The Question with an Answer Do Not Skip Any question Solve In All This Question and Explain in Detailed in simple with an Example in Exam and Technical Interview Point of view in Real Wolrd Programming Example # MongoDB Technical Interview Questions (1–100) ### Basic → Intermediate → Advanced | Real-World Focused --- ## SECTION 1: BASIC LEVEL (Q1–Q35) *Core concepts every MongoDB developer must know* ### Fundamentals 1. What is MongoDB? How is it different from a traditional relational database (SQL)? 2. What does it mean that MongoDB is a "document-oriented" (NoSQL) database? 3. What is BSON, and how is it different from JSON? 4. What is a database, collection, and document in MongoDB? How do they map to tables/rows in SQL terms? 5. What is the `_id` field? Is it required, and what happens if you don't provide one? 6. What is an `ObjectId`? What information is embedded within it (timestamp, machine, process, counter)? 7. What are the advantages of MongoDB's flexible schema? Give a real-world example (e.g., product catalog with varying attributes per category). 8. What are the disadvantages/risks of a flexible (schema-less) design in a real-world application? 9. How do you create a database and a collection in MongoDB? 10. What is the difference between `mongo`/`mongosh` shell commands and using a driver (e.g., Node.js `mongodb`/`mongoose`) in an application? ### CRUD Basics 11. How do you insert a single document and multiple documents into a collection? 12. How do you find all documents in a collection? How do you find documents matching specific criteria? 13. What is the difference between `findOne()` and `find()`? 14. How do you update a single document vs multiple documents (`updateOne` vs `updateMany`)? 15. What is the difference between `updateOne()` and `replaceOne()`? 16. How do you delete documents (`deleteOne` vs `deleteMany`)? 17. What are common update operators (`$set`, `$unset`, `$inc`, `$push`, `$pull`)? Give a real-world example of each. 18. How do you use projection to return only specific fields from a query? Why is this good practice for real-world APIs? 19. How would you query for documents where a field is greater than, less than, or between values (`$gt`, `$lt`, `$gte`, `$lte`)? 20. How would you query for documents matching multiple conditions (`$and`, `$or`)? ### Data Modeling Basics 21. What is the difference between embedding and referencing documents? Give a real-world example of each. 22. When would you embed a related document (e.g., an `Address` inside a `User`) instead of referencing it? 23. When would you reference documents instead (e.g., `Order` referencing a `User` by ID) in a real-world app? 24. What is denormalization, and why is it common in MongoDB schema design compared to SQL? 25. What is the 16MB document size limit, and how does it influence schema design decisions (e.g., a `Comments` array growing unbounded)? ### Indexing Basics 26. What is an index in MongoDB, and why does it improve query performance? 27. What is the default index MongoDB creates automatically on every collection? 28. How do you create a single-field index and a compound (multi-field) index? 29. What happens to query performance if you filter on a field that isn't indexed on a large collection? 30. What is a unique index? Give a real-world example (e.g., ensuring `email` is unique for users). ### Basic Tools & Ecosystem 31. What is MongoDB Atlas, and why would a real-world team choose it over self-hosting MongoDB? 32. What is Mongoose (in a Node.js context), and why is it commonly used instead of the raw MongoDB driver? 33. What is a Mongoose Schema, and how does it enforce structure on top of MongoDB's flexible documents? 34. What is the difference between a Mongoose "Schema" and a "Model"? 35. How would you connect a Node.js/Express application to MongoDB using Mongoose? --- ## SECTION 2: INTERMEDIATE LEVEL (Q36–Q70) *Real-world data modeling, performance, and application-level design* ### Schema Design (Real-World) 36. How would you design a schema for a real-world blog platform (`Users`, `Posts`, `Comments`)? Embed or reference the comments — why? 37. How would you design a schema for an e-commerce `Order` that captures product details at time of purchase (even if the product price later changes)? 38. What is schema versioning, and how would you handle evolving your document structure over time in a live*

*production app? 39. How would you model a many-to-many relationship in MongoDB (e.g., `Students` and `Courses`)? 40. What is the "extended reference" pattern, and when would you use it to reduce the need for extra lookups? 41. What is the "subset" pattern (storing only a subset of related data, e.g., last 10 reviews) and what real-world problem does it solve? 42. How would you design a schema to avoid unbounded array growth (e.g., a user's activity log with millions of entries over time)? 43. What is the "bucket" pattern, and when is it useful (e.g., storing IoT sensor readings grouped by time interval)? 44. How would you handle a real-world scenario where a document needs to reference data across two different collections, and you need both together frequently — index vs embed vs `$lookup`, what's the trade-off? ### Aggregation Framework 45. What is the Aggregation Pipeline in MongoDB? How is it different from a simple `find()` query? 46. What is the `$match` stage used for, and why should it typically come early in a pipeline for performance? 47. What is the `$group` stage used for? Give a real-world example (e.g., total sales per product category). 48. What is the `$project` stage used for? 49. What is `$lookup`, and how does it perform a "join" between two collections? Give a real-world example (e.g., joining `Orders` with `Users`). 50. What is `$unwind`, and when would you need it (e.g., flattening an array of order items to compute totals)? 51. What is the difference between `$match` used before vs after a `$group` stage? 52. How would you compute real-world analytics (e.g., monthly revenue, top 5 best-selling products) using the aggregation pipeline? 53. What is `$sort` combined with `$limit` used for, and why does the order of these stages matter for performance? 54. What is the difference between `$facet` and running multiple separate aggregation queries? ### Indexing (Intermediate) 55. What is a compound index, and how does field order within it affect which queries it can efficiently support? 56. What is index selectivity, and why does it matter for real-world query performance (e.g., indexing a boolean field vs a unique email field)? 57. How would you use `.explain()` to analyze whether a query is using an index (`IXSCAN`) or scanning the whole collection (`COLLSCAN`)? 58. What is a covered query, and why is it faster (no need to fetch the full document)? 59. What is a text index, and how would you implement basic full-text search on a `Products` collection? 60. What is a TTL (Time-To-Live) index, and what real-world use case does it solve (e.g., auto-expiring session tokens or OTPs)? ### Transactions & Consistency 61. Does MongoDB support multi-document ACID transactions? In what scenario would a real-world app need one (e.g., transferring funds between two accounts)? 62. What are the performance trade-offs of using multi-document transactions in MongoDB compared to relying on atomic single-document operations? 63. How does MongoDB guarantee atomicity for operations on a single document (even with nested arrays/objects)? 64. What is optimistic concurrency control, and how would you implement it in MongoDB (e.g., using a `version` field to prevent lost updates)? ### Application-Level Integration 65. How would you handle data validation at the schema level using MongoDB's JSON Schema validation, vs validating in application code (e.g., Mongoose)? 66. How would you implement pagination efficiently in MongoDB for a large collection (skip/limit vs cursor-based/range pagination)? Why does `skip()` become slow on large offsets? 67. How would you handle soft deletes (e.g., an `isDeleted` flag) vs actually removing documents, and what are the trade-offs? 68. How would you design a real-world audit trail / change history for documents that get frequently updated? 69. How would you implement search with filters, sorting, and pagination for a real-world product listing page in MongoDB? 70. What is connection pooling in the context of a Node.js app talking to MongoDB, and why does it matter under real production load? --- ## SECTION 3: ADVANCED LEVEL (Q71–Q100) *Scalability, architecture, internals, and system design thinking* ### Replication 71. What is a Replica Set in MongoDB, and why is it critical for real-world production deployments? 72. How does MongoDB elect a new Primary node if the current Primary goes down? 73. What is the Oplog, and how does it enable replication between Primary and Secondary nodes? 74. What is read preference (e.g., `primary`, `secondaryPreferred`), and when would a real-world app read from a Secondary? 75. What is write concern (e.g., `w: 1`, `w: "majority"`), and how does it affect the durability/performance trade-off in a real-world write-heavy app? 76. What is read concern, and how does it relate to consistency guarantees when reading from Secondaries? 77. What is replication lag, and what real-world problems can it cause (e.g.,*

reading stale data right after a write)? ### Sharding 78. What is sharding in MongoDB, and what real-world problem does it solve that replication alone cannot? 79. What is a shard key, and why is choosing the right one critical for a real-world sharded cluster's performance? 80. What happens if you choose a poor shard key (e.g., a monotonically increasing field like a timestamp)? What real-world issue does this cause (hot shard problem)? 81. What is chunk splitting and balancing in a sharded MongoDB cluster? 82. What is the role of `mongos` (query router) and config servers in a sharded cluster architecture? 83. How would you decide between a hashed shard key vs a ranged shard key for a real-world use case (e.g., a multi-tenant SaaS app)? ### Performance & Scaling in Production 84. How would you diagnose a slow MongoDB query in a production system (using `.explain()`, the profiler, or slow query logs)? 85. What is the MongoDB Profiler, and how would you use it to catch performance issues before they impact real users? 86. How would you handle a "hot collection" problem where a single collection receives disproportionately high read/write traffic? 87. How would you design a caching strategy (e.g., Redis) in front of MongoDB for frequently-read, rarely-changed data? 88. What is write amplification, and how can poor schema design (e.g., large arrays, frequent document growth) cause it in MongoDB? 89. How would you handle time-series data at scale in MongoDB (e.g., using the dedicated Time Series collections introduced in MongoDB 5.0)? 90. How would you architect data archiving for a real-world app where old data (e.g., orders older than 2 years) needs to move to cheaper storage without breaking the live app? ### Security 91. How would you secure a MongoDB deployment in production (authentication, authorization, network access/IP whitelisting, encryption at rest/in transit)? 92. What is Role-Based Access Control (RBAC) in MongoDB, and how would you set up least-privilege access for an application's DB user vs an admin? 93. How would you prevent NoSQL injection attacks in a real-world app where user input is used to build MongoDB queries? 94. How would you handle field-level encryption for sensitive data (e.g., SSNs, payment info) stored in MongoDB? ### Architecture & Real-World System Design 95. How would you design the MongoDB schema and indexing strategy for a real-world social media feed (posts, likes, comments, followers) that needs to scale to millions of users? 96. How would you design a multi-tenant SaaS application's data model in MongoDB (separate databases per tenant vs shared collections with a `tenantId` field)? What are the trade-offs? 97. How would you design a real-time chat application's message storage in MongoDB, considering high write volume and the need for fast retrieval of recent messages? 98. How would you architect a MongoDB-backed system for an IoT platform ingesting millions of sensor readings per day, keeping both write throughput and query performance in mind? 99. How would you plan a zero-downtime schema migration in MongoDB for a live production application (e.g., renaming a field or restructuring a nested object used by millions of documents)? 100. If you were designing the backend data layer for a real-world food delivery platform (like Zomato/Swiggy) using MongoDB — covering restaurants, menus, live orders, order history, and reviews — how would you model the collections, choose indexes, and decide what to embed vs reference, while keeping both read and write performance high at scale?

Response: **SECTION 1: BASIC LEVEL (Q1–Q35)**

1. **What is MongoDB? How is it different from a traditional relational database (SQL)?**
    - **Answer:** MongoDB is a cross-platform, document-oriented NoSQL database that stores data in dynamic, JSON-like BSON formats. Unlike SQL databases (which use rigid tables, rows, columns, and foreign-key joins), MongoDB uses collections and flexible documents, favoring embedding and horizontal scaling.
2. **What does it mean that MongoDB is a "document-oriented" (NoSQL) database?**
    - **Answer:** Data is stored as self-describing units called documents (serialized as BSON) rather than atomic rows in fixed tables, allowing nested objects and arrays to represent

complex hierarchies naturally.

3. **What is BSON, and how is it different from JSON?**
    - **Answer:** BSON (Binary JSON) is the binary serialization format used by MongoDB. Unlike text JSON, BSON includes length prefixes and adds native support for types like `ObjectId`, `Date`, and raw binary data, enabling fast machine traversal.
4. **What is a database, collection, and document in MongoDB? How do they map to tables/rows in SQL terms?**
    - **Answer:** A Database is a physical container (maps to SQL Database). A Collection is a grouping of documents (maps to a SQL Table). A Document is a single record of key-value pairs (maps to a SQL Row).
5. **What is the `_id` field? Is it required, and what happens if you don't provide one?**
    - **Answer:** The `_id` field acts as the primary key and is mandatory. If omitted, MongoDB automatically generates a unique 12-byte `ObjectId`.
6. **What is an `ObjectId`? What information is embedded within it (timestamp, machine, process, counter)?**
    - **Answer:** A 12-byte BSON type comprising a 4-byte Unix timestamp, a 5-byte machine/process hash, and a 3-byte incrementing counter, ensuring global uniqueness and chronological sortability.
7. **What are the advantages of MongoDB's flexible schema? Give a real-world example (e.g., product catalog with varying attributes per category).**
    - **Answer:** Allows disparate attributes without rigid table migrations. In a product catalog, a laptop has `RAM` and `CPU` fields, while a t-shirt has `size` and `fabric` fields.
8. **What are the disadvantages/risks of a flexible (schema-less) design in a real-world application?**
    - **Answer:** Lack of database-enforced structure can lead to inconsistent data types, application errors from unexpected null fields, and difficult querying if fields are named inconsistently.
9. **How do you create a database and a collection in MongoDB?**
    - **Answer:** Databases and collections are created implicitly when you first insert a document into them (e.g., `db.collection.insertOne(...)`).
10. **What is the difference between `mongo`/`mongosh` shell commands and using a driver (e.g., Node.js `mongodb`/`mongoose`) in an application?**
    - **Answer:** `mongosh` is an interactive CLI REPL tool for administration. Application drivers provide programmatic interfaces for backend code to connect and query.
11. **How do you insert a single document and multiple documents into a collection?**

- **Answer:** Use `db.collection.insertOne({ field: value })` for single documents and `db.collection.insertMany([{ ... }, { ... }])` for multiple documents.

12. **How do you find all documents in a collection? How do you find documents matching specific criteria?**
    - **Answer:** Use `db.collection.find()` to retrieve all documents and `db.collection.find({ status: "active" })` to filter documents matching specific criteria.
13. **What is the difference between `findOne()` and `find()`?**
    - **Answer:** `find()` returns a cursor pointing to all matching documents, whereas `findOne()` immediately returns the first single matching document as an object or `null`.
14. **How do you update a single document vs multiple documents (`updateOne` vs `updateMany`)?**
    - **Answer:** `updateOne(filter, update)` modifies only the first matching document. `updateMany(filter, update)` modifies every document matching the filter criteria.
15. **What is the difference between `updateOne()` and `replaceOne()`?**
    - **Answer:** `updateOne()` modifies specific fields using operators (like `$set`). `replaceOne()` completely overwrites the entire matched document with a new payload.
16. **How do you delete documents (`deleteOne` vs `deleteMany`)?**
    - **Answer:** `deleteOne({ status: "inactive" })` removes the first matching document. `deleteMany({ status: "inactive" })` removes all matching documents.
17. **What are common update operators (`$set`, `$unset`, `$inc`, `$push`, `$pull`)? Give a real-world example of each.**
    - **Answer:**
      - `$set`: Updates field value (`{$set: {status: "active"}}`).
      - `$unset`: Removes a field (`{$unset: {tempToken: ""}}`).
      - `$inc`: Increments numeric value (`{$inc: {loginCount: 1}}`).
      - `$push`: Adds item to array (`{$push: {tags: "sale"}}`).
      - `$pull`: Removes item from array (`{$pull: {tags: "expired"}}`).
18. **How do you use projection to return only specific fields from a query? Why is this good practice for real-world APIs?**
    - **Answer:** Pass projection arguments: `db.users.find({}, { name: 1, _id: 0 })`. This reduces network bandwidth and protects sensitive fields like passwords.

19. **How would you query for documents where a field is greater than, less than, or between values (`$gt`, `$lt`, `$gte`, `$lte`)?**
    - **Answer:** Use comparison operators inside the query object: `db.products.find({ price: { $gte: 10, $lte: 100 } })`.
20. **How would you query for documents matching multiple conditions (`$and`, `$or`)?**
    - **Answer:** Use logical operators: `db.users.find({ $or: [{ role: "admin" }, { accessLevel: { $gt: 5 } }] })`.
21. **What is the difference between embedding and referencing documents? Give a real-world example of each.**
    - **Answer:** Embedding stores related data inside the parent document (e.g., address inside user). Referencing stores related documents in separate collections and links them via ID (e.g., order referencing user ID).
22. **When would you embed a related document (e.g., an `Address` inside a `User`) instead of referencing it?**
    - **Answer:** When data has a 1-to-1 or 1-to-few relationship, is accessed together frequently, and does not grow unboundedly in size.
23. **When would you reference documents instead (e.g., `Order` referencing a `User` by ID) in a real-world app?**
    - **Answer:** When relationships are 1-to-many or many-to-many, when child data grows unboundedly, or when the child entity is updated independently.
24. **What is denormalization, and why is it common in MongoDB schema design compared to SQL?**
    - **Answer:** Denormalization duplicates data across documents to avoid costly runtime JOIN operations, optimizing read performance.
25. **What is the 16MB document size limit, and how does it influence schema design decisions (e.g., a `Comments` array growing unbounded)?**
    - **Answer:** BSON documents cannot exceed 16MB. Unbounded arrays will eventually breach this limit, necessitating the subset or bucket pattern instead of direct embedding.
26. **What is an index in MongoDB, and why does it improve query performance?**
    - **Answer:** An index is a B-tree data structure storing a sorted subset of data, allowing the query engine to locate documents without scanning the entire collection.
27. **What is the default index MongoDB creates automatically on every collection?**
    - **Answer:** An ascending unique index on the `_id` field.
28. **How do you create a single-field index and a compound (multi-field) index?**

- **Answer:** Single-field: `db.users.createIndex({ email: 1 })`. Compound: `db.orders.createIndex({ userId: 1, orderDate: -1 })`.

29. **What happens to query performance if you filter on a field that isn't indexed on a large collection?**
    - **Answer:** MongoDB performs a `COLLSCAN` (Collection Scan), inspecting every document on disk, which degrades performance as data scales.
30. **What is a unique index? Give a real-world example (e.g., ensuring `email` is unique for users).**
    - **Answer:** An index constraint preventing duplicate values. Example: `db.users.createIndex({ email: 1 }, { unique: true })`.
31. **What is MongoDB Atlas, and why would a real-world team choose it over self-hosting MongoDB?**
    - **Answer:** MongoDB Atlas is a fully-managed cloud database service providing automated backups, scaling, security hardening, and multi-region high availability.
32. **What is Mongoose (in a Node.js context), and why is it commonly used instead of the raw MongoDB driver?**
    - **Answer:** Mongoose is an ODM library offering schema validation, middleware hooks, type casting, and business logic structuring.
33. **What is a Mongoose Schema, and how does it enforce structure on top of MongoDB's flexible documents?**
    - **Answer:** A schema defines blueprint properties, data types, defaults, and validation rules at the application layer before saving to MongoDB.
34. **What is the difference between a Mongoose "Schema" and a "Model"?**
    - **Answer:** A Schema defines the data shape. A Model is a compiled constructor instance derived from the schema used to query and modify database collections.
35. **How would you connect a Node.js/Express application to MongoDB using Mongoose?**
    - **Answer:** `mongoose.connect('mongodbURI').then(() => console.log('Connected'))`.

SECTION 2: INTERMEDIATE LEVEL (Q36–Q70)

36. **How would you design a schema for a real-world blog platform (`Users`, `Posts`, `Comments`)? Embed or reference the comments — why?**
    - **Answer:** Use referencing for comments if posts can have thousands of comments to avoid unbounded array growth and the 16MB document size limit.
37. **How would you design a schema for an e-commerce `Order` that captures product details at time of purchase (even if the product price later changes)?**

- **Answer:** Embed a snapshot array of purchased items (including name, historical price paid, and quantity) directly inside the `Order` document.

38. **What is schema versioning, and how would you handle evolving your document structure over time in a live production app?**
    - **Answer:** Include a `schemaVersion` integer field on documents and let application code dynamically handle or lazily migrate legacy structures upon retrieval.
39. **How would you model a many-to-many relationship in MongoDB (e.g., `Students` and `Courses`)?**
    - **Answer:** Store an array of course ObjectIds inside the `Student` document, or use a separate junction collection if both arrays grow unboundedly.
40. **What is the "extended reference" pattern, and when would you use it to reduce the need for extra lookups?**
    - **Answer:** Duplicating a few frequently accessed fields from a referenced document (e.g., storing `username` inside a `Post`) to eliminate `$lookup` joins.
41. **What is the "subset" pattern (storing only a subset of related data, e.g., last 10 reviews) and what real-world problem does it solve?**
    - **Answer:** Storing only the most relevant subset of child documents inside the main document, preventing 16MB limit violations and optimizing RAM usage.
42. **How would you design a schema to avoid unbounded array growth (e.g., a user's activity log with millions of entries over time)?**
    - **Answer:** Use the Bucket Pattern or shift to a separate time-series logging collection where each entry is its own document.
43. **What is the "bucket" pattern, and when is it useful (e.g., storing IoT sensor readings grouped by time interval)?**
    - **Answer:** Grouping multiple sequential events into a single document bucket, reducing index overhead and total document counts.
44. **How would you handle a real-world scenario where a document needs to reference data across two different collections, and you need both together frequently — index vs embed vs `$lookup`, what's the trade-off?**
    - **Answer:** Embedding optimizes reads but duplicates data; `$lookup` keeps data normalized but adds query overhead; indexed references balance storage and join performance.
45. **What is the Aggregation Pipeline in MongoDB? How is it different from a simple `find()` query?**
    - **Answer:** A data processing pipeline of stages (`$match`, `$group`, `$project`) that transform and compute analytics, whereas `find()` merely retrieves documents.

46. **What is the `$match` stage used for, and why should it typically come early in a pipeline for performance?**
    - **Answer:** Filters documents. Placing it early reduces data volume through subsequent stages and allows index utilization.
47. **What is the `$group` stage used for? Give a real-world example (e.g., total sales per product category).**
    - **Answer:** Aggregates documents by key (`{$group: { _id: "$category", totalSales: { $sum: "$amount" } }}`).
48. **What is the `$project` stage used for?**
    - **Answer:** Reshapes documents by renaming, adding, or excluding specific fields.
49. **What is `$lookup`, and how does it perform a "join" between two collections? Give a real-world example (e.g., joining `Orders` with `Users`).**
    - **Answer:** Performs a left outer join (`{$lookup: {from: "users", localField: "userId", foreignField: "_id", as: "user"}}`).
50. **What is `$unwind`, and when would you need it (e.g., flattening an array of order items to compute totals)?**
    - **Answer:** Deconstructs an array field from input documents to output a separate document for each element.
51. **What is the difference between `$match` used before vs after a `$group` stage?**
    - **Answer:** `$match` before `$group` filters raw documents using indexes. `$match` after `$group` filters the aggregated summary results.
52. **How would you compute real-world analytics (e.g., monthly revenue, top 5 best-selling products) using the aggregation pipeline?**
    - **Answer:** Chain `$match`, `$group`, `$sort`, and `$limit` stages.
53. **What is `$sort` combined with `$limit` used for, and why does the order of these stages matter for performance?**
    - **Answer:** Used for top-N queries. Proper indexing allows MongoDB to optimize sorting and limiting without sorting the entire dataset in memory.
54. **What is the difference between `$facet` and running multiple separate aggregation queries?**
    - **Answer:** `$facet` processes multiple aggregation pipelines concurrently within a single stage on the same input documents.
55. **What is a compound index, and how does field order within it affect which queries it can efficiently support?**

- **Answer:** An index on multiple fields. Field order matters because it supports queries filtering on prefix subsets (ESR rule).

56. **What is index selectivity, and why does it matter for real-world query performance (e.g., indexing a boolean field vs a unique email field)?**
    - **Answer:** Selectivity measures how uniquely an index filters documents. High selectivity (email) reduces search space; low selectivity (boolean) is inefficient.
57. **How would you use `.explain()` to analyze whether a query is using an index (`IXSCAN`) or scanning the whole collection (`COLLSCAN`)?**
    - **Answer:** Run `db.collection.find({query}).explain("executionStats")` to verify execution metrics.
58. **What is a covered query, and why is it faster (no need to fetch the full document)?**
    - **Answer:** A query where all requested fields exist in the index, allowing results to be returned directly without reading document data from disk.
59. **What is a text index, and how would you implement basic full-text search on a `Products` collection?**
    - **Answer:** Specialized index supporting text search (`db.products.createIndex({ description: "text" })`).
60. **What is a TTL (Time-To-Live) index, and what real-world use case does it solve (e.g., auto-expiring session tokens or OTPs)?**
    - **Answer:** An index that automatically deletes documents after a specified time or date, ideal for sessions and OTPs.
61. **Does MongoDB support multi-document ACID transactions? In what scenario would a real-world app need one (e.g., transferring funds between two accounts)?**
    - **Answer:** Yes, across replica sets. Needed for complex financial transfers requiring atomicity across multiple collections.
62. **What are the performance trade-offs of using multi-document transactions in MongoDB compared to relying on atomic single-document operations?**
    - **Answer:** Transactions consume more memory, incur locking overhead, and lower write throughput.
63. **How does MongoDB guarantee atomicity for operations on a single document (even with nested arrays/objects)?**
    - **Answer:** All modifications to a single document are atomic at the storage engine level.
64. **What is optimistic concurrency control, and how would you implement it in MongoDB (e.g., using a `version` field to prevent lost updates)?**

- **Answer:** Checking if a version field matches before updating (`{$set: {data}, $inc: {__v: 1}}, {query: {_id, __v: version}}`).

65. **How would you handle data validation at the schema level using MongoDB's JSON Schema validation, vs validating in application code (e.g., Mongoose)?**
    - **Answer:** JSON Schema enforces rules at the database storage layer for all clients, while Mongoose validation occurs at application runtime.
66. **How would you implement pagination efficiently in MongoDB for a large collection (skip/limit vs cursor-based/range pagination)? Why does `skip()` become slow on large offsets?**
    - **Answer:** `skip()` scans and discards records sequentially. Cursor-based pagination uses indexed fields (e.g., `_id > lastId`) for O(1) performance.
67. **How would you handle soft deletes (e.g., an `isDeleted` flag) vs actually removing documents, and what are the trade-offs?**
    - **Answer:** Soft deletes preserve audit trails and allow restoration, but require filtering `isDeleted: false` across queries and consume storage.
68. **How would you design a real-world audit trail / change history for documents that get frequently updated?**
    - **Answer:** Use MongoDB Change Streams to listen to oplog events or maintain a separate `AuditLogs` collection.
69. **How would you implement search with filters, sorting, and pagination for a real-world product listing page in MongoDB?**
    - **Answer:** Combine `$match`, compound indexing matching filter/sort fields, and range-based pagination.
70. **What is connection pooling in the context of a Node.js app talking to MongoDB, and why does it matter under real production load?**
    - **Answer:** Maintaining a pool of open TCP connections, avoiding connection overhead for incoming requests.

SECTION 3: ADVANCED LEVEL (Q71–Q100)

71. **What is a Replica Set in MongoDB, and why is it critical for real-world production deployments?**
    - **Answer:** A group of mongod instances maintaining the same dataset (Primary and Secondaries) ensuring high availability and automatic failover.
72. **How does MongoDB elect a new Primary node if the current Primary goes down?**
    - **Answer:** Secondaries notice missed heartbeats and hold a consensus election based on priority and oplog recency.

73. **What is the Oplog, and how does it enable replication between Primary and Secondary nodes?**
    - **Answer:** A capped collection recording all write operations, replayed asynchronously by Secondary nodes.
74. **What is read preference (e.g., `primary`, `secondaryPreferred`), and when would a real-world app read from a Secondary?**
    - **Answer:** Settings directing read operations. Reading from Secondaries offloads read traffic for analytics.
75. **What is write concern (e.g., `w: 1`, `w: "majority"`), and how does it affect the durability/performance trade-off in a real-world write-heavy app?**
    - **Answer:** Specifies acknowledgment levels required before returning write success (`w: 1` is fast; `w: "majority"` ensures durability across nodes).
76. **What is read concern, and how does it relate to consistency guarantees when reading from Secondaries?**
    - **Answer:** Determines read isolation levels, protecting against reading uncommitted data during failovers.
77. **What is replication lag, and what real-world problems can it cause (e.g., reading stale data right after a write)?**
    - **Answer:** The delay between a write on the Primary and replication to Secondaries, potentially causing stale reads.
78. **What is sharding in MongoDB, and what real-world problem does it solve that replication alone cannot?**
    - **Answer:** Horizontal data partitioning across multiple machines, solving storage capacity and throughput bottlenecks.
79. **What is a shard key, and why is choosing the right one critical for a real-world sharded cluster's performance?**
    - **Answer:** An indexed field determining document distribution across shards. Poor choices cause uneven data distribution.
80. **What happens if you choose a poor shard key (e.g., a monotonically increasing field like a timestamp)? What real-world issue does this cause (hot shard problem)?**
    - **Answer:** All new writes target a single shard handling the current time range, causing a hot shard bottleneck.
81. **What is chunk splitting and balancing in a sharded MongoDB cluster?**
    - **Answer:** Chunks are ranges of shard key values. When large, the balancer splits and migrates chunks across shards.

82. **What is the role of `mongos` (query router) and config servers in a sharded cluster architecture?**
    - **Answer:** `mongos` routes client queries to appropriate shards. Config servers store cluster metadata and chunk routing tables.
83. **How would you decide between a hashed shard key vs a ranged shard key for a real-world use case (e.g., a multi-tenant SaaS app)?**
    - **Answer:** Hashed keys distribute writes evenly; ranged keys optimize queries targeting specific value ranges or tenants.
84. **How would you diagnose a slow MongoDB query in a production system (using `.explain()`, the profiler, or slow query logs)?**
    - **Answer:** Enable the database profiler, check slow query logs, and run `.explain("executionStats")` to identify `COLLSCAN` queries.
85. **What is the MongoDB Profiler, and how would you use it to catch performance issues before they impact real users?**
    - **Answer:** A built-in diagnostic tool recording slow operations exceeding latency thresholds into `system.profile`.
86. **How would you handle a "hot collection" problem where a single collection receives disproportionately high read/write traffic?**
    - **Answer:** Shard the collection using an optimal shard key, add read replicas, or add a caching layer (Redis).
87. **How would you design a caching strategy (e.g., Redis) in front of MongoDB for frequently-read, rarely-changed data?**
    - **Answer:** Cache query results in Redis with TTL or invalidation triggers on write operations.
88. **What is write amplification, and how can poor schema design (e.g., large arrays, frequent document growth) cause it in MongoDB?**
    - **Answer:** When minor updates force the storage engine to rewrite entire large documents, causing disk fragmentation.
89. **How would you handle time-series data at scale in MongoDB (e.g., using the dedicated Time Series collections introduced in MongoDB 5.0)?**
    - **Answer:** Use native Time Series collections which automatically optimize storage layout in compressed columnar formats.
90. **How would you architect data archiving for a real-world app where old data (e.g., orders older than 2 years) needs to move to cheaper storage without breaking the live app?**
    - **Answer:** Use MongoDB Atlas Online Archive or write cron workers to migrate old documents to cold storage tiers.

91. **How would you secure a MongoDB deployment in production (authentication, authorization, network access/IP whitelisting, encryption at rest/in transit)?**
    - **Answer:** Enable TLS for transport encryption, KMS for storage encryption, SCRAM authentication, and IP whitelisting.
92. **What is Role-Based Access Control (RBAC) in MongoDB, and how would you set up least-privilege access for an application's DB user vs an admin?**
    - **Answer:** RBAC assigns specific roles granting minimum necessary permissions required for application operational scope.
93. **How would you prevent NoSQL injection attacks in a real-world app where user input is used to build MongoDB queries?**
    - **Answer:** Sanitize input types, avoid passing raw request objects directly into filters, and use ODM validation.
94. **How would you handle field-level encryption for sensitive data (e.g., SSNs, payment info) stored in MongoDB?**
    - **Answer:** Use Client-Driven Field Level Encryption (CSFLE) to encrypt sensitive fields on the application side.
95. **How would you design the MongoDB schema and indexing strategy for a real-world social media feed (posts, likes, comments, followers) that needs to scale to millions of users?**
    - **Answer:** Use fan-out models and compound indexes like `{ authorId: 1, createdAt: -1 }`.
96. **How would you design a multi-tenant SaaS application's data model in MongoDB (separate databases per tenant vs shared collections with a `tenantId` field)? What are the trade-offs?**
    - **Answer:** Shared collections with `tenantId` scale better for thousands of tenants, whereas separate databases offer isolated backups.
97. **How would you design a real-time chat application's message storage in MongoDB, considering high write volume and the need for fast retrieval of recent messages?**
    - **Answer:** Store messages in collections indexed by `{ chatId: 1, createdAt: -1 }` with capped sizes or archiving.
98. **How would you architect a MongoDB-backed system for an IoT platform ingesting millions of sensor readings per day, keeping both write throughput and query performance in mind?**
    - **Answer:** Use Time Series collections with hashed or time-range sharding.
99. **How would you plan a zero-downtime schema migration in MongoDB for a live production application (e.g., renaming a field or restructuring a nested object used by millions of documents)?**

- **Answer:** Implement dual-read/dual-write logic supporting both schemas, migrate historical records via background workers, and then deprecate old fields.

100. **If you were designing the backend data layer for a real-world food delivery platform using MongoDB — covering restaurants, menus, live orders, order history, and reviews — how would you model the collections, choose indexes, and decide what to embed vs reference?**

    - **Answer:** Embed static menus inside restaurants; use dedicated `Orders` collections with geospatial `2dsphere` indexes for live tracking; reference user histories and reviews using subset patterns for high scalability.
